

Aarya Arban

T11 – 05

Assignment No. 3

AIM – Implement the concepts of Access Modifiers, Union, Tuples using Typescript

LO2 - Understand the basic concepts of Typescript for designing web applications

THEORY:

1. Access Modifiers

Access Modifiers in TypeScript define the visibility of class members

(variables or methods). There are three main types: Modifier Description

public Accessible from anywhere (default). private Accessible only
inside the class.

protected Accessible inside the class and its subclasses.

2. Union Types

A Union allows a variable to hold more than one type. Useful when you want to accept multiple types for flexibility.

3. Tuples

A Tuple is a fixed-length array with elements of specific types. It allows storing values of different types together in a specific order.

PROGRAM:

```
// AccessModifiers.ts class

Person {  private name:
string;  protected age:
number;  public city: string;  constructor(name:
string, age: number, city: string) {      this.name = name;
this.age = age;      this.city = city;
    }
    public getDetails(): string {      return `Name: ${this.name}, Age:
${this.age}, City: ${this.city}`;
    }
} class Employee extends Person {  private employeeId: number;
constructor(name: string, age: number, city: string, employeeId:
number) {      super(name, age, city);      this.employeeId =
employeeId;
    }
    public getEmployeeDetails(): string {      return `${this.getDetails()},
Employee ID: ${this.employeeId}`;
    }
}

// Testing access modifiers and error handling const emp = new
Employee("John Doe", 30, "New York", 12345);

try {
```

```
// Trying to access private member    console.log(emp.name); //
```

Error

```
} catch (error) {    console.log("Error accessing private member
```

```
'name':", error.message);
```

```
} try
```

```
{
```

```
// Trying to access protected member    console.log(emp.age); //
```

Error

```
} catch (error) {    console.log("Error accessing protected member
```

```
'age':", error.message);
```

```
}
```

```
console.log(emp.getEmployeeDetails()); // Accessing public method OUTPUT
```

```
\\
```



The screenshot shows the OneCompiler web IDE interface. The editor displays a TypeScript file named `HelloWorld.ts` with the following code:

```
1 // AccessModifiers.ts
2
3 class Person {
4     private name: string;
5     protected age: number;
6     public city: string;
7
8     constructor(name: string, age: number, city: string) {
9         this.name = name;
10        this.age = age;
11        this.city = city;
12    }
13
14    public getDetails(): string {
15        return `Name: ${this.name}, Age: ${this.age}, City: ${this.city}`;
16    }
17 }
18
19 class Employee extends Person {
20     private employeeId: number;
21
22     constructor(name: string, age: number, city: string, employeeId: number) {
23         super(name, age, city);
24         this.employeeId = employeeId;
25     }
26
27     public getEmployeeDetails(): string {
28         return `${this.getDetails()}, Employee ID: ${this.employeeId}`;
29     }
30 }
```

The output pane on the right shows the following errors:

```
script.ts(38,21): error TS2341: Property 'name' is private and only accessible within class 'Person'.
script.ts(45,21): error TS2445: Property 'age' is protected and only accessible within class 'Person' and its subclasses.
```

```
// UnionAndTuples.ts
```

```
// Union Type Example with multiple types function printIdentifier(id:
```

```
number | string | boolean): void {  if (typeof id === 'number') {
```

```
  console.log(`ID (number): ${id}`);  } else if (typeof id === 'string') {
```

```
  console.log(`ID (string): ${id}`);
```

```
    } else {      console.log(`ID
```

```
(boolean): ${id}`);
```

```
  }
```

```
}
```

```
// Tuple Example with multiple data types let user:
```

```
[number, string, boolean, string?]; user = [1, "Alice",
```

```
true, "Developer"]; console.log("Tuple Example:",
```

```
user);
```

```
// Testing the union type function with different
```

```
types printIdentifier(101); printIdentifier("XYZ-
```

```
123"); printIdentifier(true);
```

OUTPUT:

[PRICING](#)

HelloWorld.ts43dj4xugr

```
1 // UnionAndTuples.ts
2
3 // Union Type Example with multiple types
4 function printIdentifier(id: number | string) {
5     if (typeof id === 'number') {
6         console.log(`ID (number): ${id}`);
7     } else if (typeof id === 'string') {
8         console.log(`ID (string): ${id}`);
9     } else {
10        console.log(`ID (boolean): ${id}`);
11    }
12 }
13
14 // Tuple Example with multiple data types
15 let user: [number, string, boolean, string?];
16 user = [1, "Alice", true, "Developer"];
17
18 console.log("Tuple Example:", user);
19
20 // Testing the union type function with differ
21 printIdentifier(101);
22 printIdentifier("XYZ-123");
23 printIdentifier(true);
```

STDIN

Input for the program (Optional)

Output:

Tuple Example: [1, 'Alice', true, 'Developer']

ID (number): 101

ID (string): XYZ-123

ID (boolean): true